Сценарий 1. `RLocalCachedMap`: двухуровневый кэш и инвалидация.

Тест: `src/test/java/io/github/olaaa/redissoncachelab/scenario/Scenario1LocalCachedMapTest.java`. Сервис: `LocalCachedMapProductService` (карта `productsLocalCached`, `LOCAL_TTL` = 30 секунд). Клиент создаёт `redisson-spring-boot-starter`. Ручная проверка: `http/01-local-cached-map.http` — на двух запущенных репликах.

Ситуация.

В distributed-cache-lab двухуровневый кэш и инвалидация между репликами собраны вручную: Caffeine перед Redis (сценарий 3) и рассылка через Redis Pub/Sub (сценарий 4). Это около двухсот строк, и в них легко ошибиться: порядок операций, подписка, страховка на случай потерянного сообщения.

В Redisson всё это уже есть: `RLocalCachedMap` — карта в Redis с локальной копией в памяти каждого экземпляра и автоматической инвалидацией через Pub/Sub. Сценарий показывает ту же задачу на готовом механизме: что он делает сам, как его настроить и чего в бесплатной версии нет.

Подключение.

Redisson подключён через `redisson-spring-boot-starter`. Своей конфигурации клиента в проекте нет: стартер сам создаёт бины `RedissonClient`, `RedissonReactiveClient` и `RedissonRxClient`.

Адрес Redis стартер берёт там же, где Spring Boot для Spring Data Redis: из `DataRedisConnectionDetails` (в исходнике `RedissonAutoConfigurationV4` — `getStandalone`, `getSentinel`, `getCluster`), а без них — из свойств `spring.data.redis.*`. Поэтому адрес подставляется сам и из `compose.yaml` при `mvn spring-boot:run`, и из контейнера Testcontainers с `@ServiceConnection(name = "redis")` в тестах. Для тонкой настройки клиента есть свойство `spring.redis.redisson.config` (или `file`) с YAML-настройкой Redisson.

Стартер приносит и Spring Data Redis — без Lettuce и Jedis: фабрикой соединений Spring Data Redis становится `RedissonConnectionFactory`. Второго клиента Redis в проекте нет, поэтому сценарии с Redisson и живут в отдельном репозитории: в distributed-cache-lab Spring Data Redis работает через Lettuce.

Один `RedissonClient` — одна реплика: у клиента свои соединения с Redis, в том числе отдельное для Pub/Sub. В тестах вторая реплика — второй `RedissonClient` с копией настройки бина: `Redisson.create(new Config(redisson.getConfig()))`.

Как это работает.

```
LocalCachedMapOptions<Long, Product> options = LocalCachedMapOptions.<Long,
Product>name(MAP_NAME)
        .codec(new TypedJsonJackson3Codec(Long.class, Product.class))
        .cacheSize(10_000)
        .evictionPolicy(EvictionPolicy.LRU)
        .timeToLive(LOCAL_TTL)
        .syncStrategy(SyncStrategy.INVALIDATE)
        .reconnectionStrategy(reconnectionStrategy);
this.products = redisson.reactive().getLocalCachedMap(options);
```

- **`getLocalCachedMap` у реактивного клиента** возвращает `RLocalCachedMap Reactive` — методы возвращают `Mono`, сервис не блокирует потоки WebFlux.
- **`codec`** — как ключи и значения превращаются в байты. `TypedJsonJackson3 Codec` пишет JSON на Jackson 3, который и так приходит со Spring Boot 4; в Redisson 4.8 есть и вариант на Jackson 2 (`TypedJsonJacksonCodec`), но ему нужен Jackson 2 в зависимостях.
- **`cacheSize`**, **`evictionPolicy`**, **`timeToLive`** — размер, вытеснение и срок жизни локальной копии, как `maximumSize` и `expireAfterWrite` у Caffeine.
- **`syncStrategy(INVALIDATE)`** — при изменении записи остальные экземпляры с тем же именем удаляют свою копию. Значение по умолчанию.
- **`reconnectionStrategy`** — что делать после обрыва связи (ниже).

Чтение и изменение:

```
public Mono<Product> findById(long id) {
    return products.get(id)
            .switchIfEmpty(Mono.defer(() -> source.load(id)
                    .flatMap(product -> products.fastPut(id, product).thenReturn(pr
oduct))));
}

public Mono<Boolean> changePrice(long id, BigDecimal price) {
    return repository.updatePrice(id, price)
            .flatMap(updatedRows -> updatedRows == 0
                    ? Mono.just(false)
                    : products.fastRemove(id).thenReturn(true));
}
```

`get` сначала смотрит в локальную копию и только при промахе отправляет в Redis `HGET`. `fastPut` пишет в хеш и рассылает событие остальным экземплярам, `fastRemove` удаляет из хеша, из своей копии и рассылает инвалидацию. Код рассылки и подписки — внутри Redisson. Всё, что в distributed-cache-lab было в `BroadcastInvalidationProductService`, здесь — одна настройка и вызов `fastRemove`.

В Redis карта — обычный хеш: ключ `productsLocalCached`, поле — id товара, значение — JSON. Документация Redisson: «Instances with the same name share a pub/sub channel. This channel is used for exchanging of update/invalidate events between all instances».

Обрыв связи: `ReconnectionStrategy`

Инвалидации идут через Pub/Sub, а Pub/Sub доставляет сообщение не больше одного раза: экземпляр, который был отключён, сообщение потеряет (сценарий 4 distributed-cache-lab). Redisson решает это настройкой. Варианты из исходника `LocalCachedMapOptions` (Redisson 4.8.0):

- **`NONE`** — «No reconnect handling». Значение по умолчанию. Копия после обрыва остаётся, пропущенные инвалидации не учтены.
- **`CLEAR`** — «Clear local cache if map instance disconnected». После переподключения локальная копия очищается целиком — то, что документация Redis советует для клиентских кэшей: «Make sure that if the connection is lost, the local cache is flushed».
- **`LOAD`** — «Store invalidated entry hash in invalidation log for 10 minutes. Cache keys for stored invalidated entry hashes will be removed if LocalCachedMap instance has been disconnected less than 10 minutes or whole local cache will be cleaned otherwise». Экземпляр после короткого обрыва удаляет только те записи, которые за это время менялись.

По умолчанию стоит `NONE`, поэтому стратегию нужно выбрать явно. В сервисе — `CLEAR`.

Чего нет в бесплатной версии

- **Срок жизни записей в Redis.** У хеша `productsLocalCached` нет срока жизни, и у отдельных записей тоже: в Redis значение живёт, пока его не удалят. Карта с локальным кэшем и сроком жизни каждой записи — `RLocalCachedMapCache` — в открытой версии не реализована, `getLocalCachedMapCache` бросает `UnsupportedOperationException: This feature is implemented in the`

`Redisson PRO version`. Тест это проверяет. Значит, устаревшее значение в Redis исправит только явное удаление: `fastRemove` при изменении или `expire` на весь хеш.

- **Spring Cache с локальным кэшем.** Менеджеры `@Cacheable` с локальным кэшем (`RedissonSpringLocalCachedCacheManager` и родственные) по документации Redisson — только в PRO. В открытой версии `RLocalCachedMap` используется напрямую, как в этом сервисе.

Что проверяют тесты.

`repeatedReadsAreServedLocallyWithoutRedis`: десять чтений — одно обращение к источнику и одна команда `HGET` в Redis по `INFO commandstats`. Девять повторных отдала локальная копия, без сети.

`anotherReplicaGetsValueFromRedisNotFromSource`: реплика B — второй `RedissonClient` — находит товар в хеше Redis, обращений к источнику по-прежнему одно.

`priceChangeOnOneReplicaInvalidatesLocalCopyOnAnother`: обе реплики прочитали товар, A меняет цену. Без единой строки кода рассылки реплика B удаляет свою копию и отдаёт новую цену 1990.00. Обращений к источнику — два.

`reconnectionStrategyClearDropsLocalCopyAfterDisconnect`: две реплики со стратегиями `CLEAR` и `NONE` прочитали товар. Команда `CLIENT KILL TYPE pubsub` закрывает в Redis все соединения Pub/Sub — как обрыв сети. Реплика с `CLEAR` после переподключения очистила свою копию, реплика с `NONE` — сохранила.

`redisSideIsHashWithoutPerEntryTimeToLiveInOpenSourceVersion`: в Redis — хеш, значение по полю `1` — `{"id":1,"name":"Чайник","price":2490.00}`, `TTL` хеша — `-1`, а `getLocalCachedMapCache` бросает исключение про Redisson PRO.

Тесты смотрят в Redis в обход Redisson — командами `redis-cli` внутри контейнера Testcontainers.

Попробовать вручную.

Запустить две реплики в двух терминалах:

```
mvn spring-boot:run
mvn spring-boot:run -Dspring-boot.run.arguments=--server.port=8081
```

`http/01-local-cached-map.http`: сброс, товар 1 на обеих репликах, изменение цены на 8080 — и 8081 сразу отдаёт новую цену. Посмотреть хеш в Redis:

```
docker compose exec redis redis-cli HGETALL productsLocalCached
docker compose exec redis redis-cli TTL productsLocalCached
```

Сравнение с ручной реализацией.

Что.	distributed-cache-lab, сценарии 3–4.	`RLocalCachedMap`.
Локальная копия.	Caffeine, `expireAfterWrite`.	своя или Caffeine (`cacheProvider`), `timeToLive`.
Рассылка инвалидации.	свой код: `PUBLISH` после удаления из Redis.	сама при `fastPut` и `fastRemove`.
Потерянная инвалидация.	только короткий `LOCAL_TTL`.	`ReconnectionStrategy.CLEAR` или `LOAD` плюс `timeToLive`.
Срок жизни записи в Redis.	`SET … EX` на каждый ключ.	нет в открытой версии.
`@Cacheable`.	нет (вручную).	нет в открытой версии.
Клиент.	Spring Data Redis и Lettuce.	Redisson через `redisson-spring-boot-starter`.

Как сказать на собеседовании

«Двухуровневый кэш с инвалидацией между репликами можно написать самой — Caffeine, Redis и рассылка через Pub/Sub, — а можно взять готовый `RLocalCachedMap` из Redisson: локальная копия в каждом экземпляре, хеш в Redis, инвалидация через Pub/Sub сама при изменении. Обязательно выбираю `ReconnectionStrategy`: по умолчанию `NONE`, и после обрыва связи копия может остаться устаревшей; `CLEAR` очищает её после переподключения. Ограничения открытой версии: нет срока жизни записей в Redis — это `RLocalCachedMapCache` в PRO — и нет `@Cacheable` с локальным кэшем».

Источники

- Redisson — Collections: Map, local cache, Integration with Spring.
- `redisson-spring-boot-starter` 4.8.0, байткод: `RedissonAutoConfigurationV4`.
- Redisson 4.8.0, исходники: `LocalCachedMapOptions`, `RedissonReactiveClient.getLocalCachedMap`, `RLocalCachedMapReactive`, `Redisson.getLocalCachedMapCache`, `TypedJsonJackson3Codec`.
- Redis — Pub/Sub: Delivery semantics, Client-side caching.